

Module 13: Deadlock Prevention & Havender's Algorithm

1. Prevention Strategies & Resource Ordering Trace

Invalidating the 4 Coffman Conditions

1. Mutual Exclusion Denial

- Make resources shareable (Read-only files). Spooling for printers.
- Impractical for inherently non-shareable resources (Mutexes, Tape).

2. Hold & Wait Denial (Conservative Protocol)

- Protocol A: Request ALL required resources before execution starts.
- Protocol B: Release all held resources before requesting new ones.
- Severe Drawback: Extremely low resource utilization & starvation!

3. No Preemption Denial

- If P holding R1 asks for unavailable R2 → R1 is forcibly preempted!
- Only feasible for saveable states (CPU registers, RAM pages), not disks.

4. Circular Wait Denial (Havender's Algorithm)

- Imposes global strict linear ordering on all resources.
- $F: R \rightarrow N$. Process can ONLY request in strictly increasing order!

Havender's Protocol (Gateway Slide 230 Trace)

Mathematical Proof of Circular Wait Denial:

If a cycle existed: $F(R0) < F(R1) < F(R2) < \dots < F(Rn) < F(R0)$.
This implies $F(R0) < F(R0)$, which is a mathematical IMPOSSIBILITY!
⇒ Circular Wait can NEVER form in any state under Havender's rule.
⇒ 100% Deadlock Free by Design!

2. The 4 Elimination Protocols

Coffman Condition	Elimination Protocol	Trade-off / Penalty
Mutual Exclusion	Spooling / Shared read-only	Not possible for dedicated hardware
Hold and Wait	Request all resources at start	Very poor resource utilization & starvation
No Preemption	Forcibly revoke held resources	State corruption risk; limited to CPU/RAM
Circular Wait	Havender's strict linear ordering	Best practical method ; programmer discipline needed